

NodeJS / Final Server – Exercise 9

In this exercise, you'll create a platform to store documents. The platform provides a simple way to create, update, and view documents, and to explore each document's history. The data will be stored in MongoDB collections.

The documents are structured in folders. To simplify the structure, each document will be saved with its **path**. A common path prefix means that the documents are in the same folder.

Each endpoint accessed on the server should include a header named **X-User-Id** with the user ID. This ID identifies the author of a document and the person who updated it. (The users are not stored on the server and can have any ID).

Documents

Data models:

- **Document:**
 - **id** – string
 - **author** – string (user ID)
 - **path** – string
 - **title** – string
 - **content** – string
 - **createdAt** – Date
 - **lastUpdatedAt** – string
 - **lastUpdatedBy** – string (user ID)
- **Document details:**
 - **id** – string
 - **author** – string (user ID)
 - **path** – string
 - **title** – string

Endpoints:

- **Create a document:**
 - **Input** (body): path, title, content.
 - **Returns:** the newly created document details (only the details of the document without the content).
- **Get all documents** – Returns a list of the documents' details. Optional query parameters (A request can contain more than one query parameter):
 - **pathPrefix:** Returns all the documents in a specific folder.
 - **sortBy:** Sorts the documents by the given field name. If the field name starts with "-", the sort will be descending. For example: `/api/documents?sortBy=-lastUpdatedAt`.
 - **author:** Returns only the documents of a specific author.

- Get a document by its ID.
- Update a document – Returns the updated document details.
- Delete a document – Returns the deleted document details.
- Download the document as a PDF file. The PDF file includes the document title and content.

History

Data model:

- **Operation:**
 - **user** – string (user ID)
 - **documentId** – string
 - **documentPath** – string
 - **documentAuthor** – string
 - **timestamp** – Date
 - **operationType** – string (can be **CREATE**, **UPDATE**, or **DELETE**)

Endpoints:

- List operations – Returns a paginated list of the operations, sorted descending by the operation timestamp. This endpoint always returns paginated data with default paging parameters. Optional query parameters (A request can contain more than one query parameter):
 - **pathPrefix**: Returns the history of a specific folder or file.
 - **fieldName**: Filter the results by the given field name and value. The supported filtering fields are **user**, **documentId**, **documentAuthor**, and **operationType**. For example: **/api/history?user=2153&operationType=DELETE**.
- Clear history – Clears the history collection.

Functionality

- Every HTTP request to the server should include the user ID in the **X-User-Id** header. A request without the header will be rejected with a 401 (Unauthorized) status code. User ID validation and parsing will be handled by a common middleware that adds the **userId** to the request object.
- Every operation on a document, including create, update, and delete, should be stored as an operation in the history collection. Implement this behavior in middleware that runs after each relevant operation (Inject the history service into the documents API and into the middleware as parameters). This middleware can read the HTTP response to get the document details.
- The body of the requests should be validated. Implement the validation in a middleware.
- Query parameters should not be validated. Ignore them if they're not valid.
- Implement an **Error middleware** to catch all the errors that are thrown during the requests. This middleware should return an appropriate user-facing error and log the actual error to the console. The logged error message should include all the relevant details needed to investigate it later.
- Return an appropriate status code for every case, with an explainable status message.

Notes

- Organize the code, including all the layers – api, service, and DAL. Put utils, middlewares, and types in the relevant files.
- Run the app, test all the endpoints from Postman, and submit the Postman collection file with the project.
- Make sure to write clean code that follows all conventions.